

Large Language Model Architectures: A Technical Presentation

For the expert programmer entering the LLM field. March 2026.

Part 1: The Problem

What Does It Mean to Process Language Computationally?

Language is sequential. Words arrive one after another, and the meaning of each depends on what came before -- sometimes on what came hundreds or thousands of words before. The computational challenge is deceptively simple to state: given a sequence of words, predict what comes next. Everything in this document flows from that problem.

Before 2017, the dominant approach to sequence modeling was the **Recurrent Neural Network** (RNN). An RNN processes tokens one at a time, maintaining a hidden state vector that updates at each step: $h_t = f(h_{t-1}, x_t)$. This creates a sequential dependency chain. You cannot compute h_t without first computing h_{t-1} .

Two refinements -- **LSTMs** (Hochreiter & Schmidhuber, 1997) and **GRUs** (Cho et al., 2014) -- added gating mechanisms to control information flow, partially solving the *vanishing gradient problem* where gradients shrink exponentially over long sequences. But two fundamental bottlenecks remained:

1. **Sequential processing.** Each token depends on the previous one, preventing parallelization across the sequence dimension. Training time scales linearly with sequence length. You cannot use your 8 GPUs to process 8 positions simultaneously -- you must wait for each step to finish before starting the next.
2. **Fixed-size bottleneck.** In encoder-decoder RNNs, the entire input must be compressed into a single fixed-length hidden vector before decoding begins. Think of it as trying to summarize a 10,000-word document into a single 1024-dimensional vector and then reconstructing it. Long inputs lose information catastrophically.

The first crack in the wall came from Bahdanau, Cho & Bengio (2014), who introduced **attention** to sequence-to-sequence models. Instead of forcing the encoder to compress everything into one vector, the decoder could look back at all encoder hidden states and weight them by relevance at each generation step. This eliminated the fixed-length bottleneck and dramatically improved translation quality on long sentences.

The key conceptual move -- letting the model *attend* selectively to different parts of the input -- became the foundation of everything that followed.

But the sequential bottleneck remained. You still processed tokens one at a time. The question was: how do you let every word see every other word, without waiting?

Part 2: The Transformer Solution

Attention as a Database Lookup

In June 2017, Vaswani et al. published "Attention Is All You Need," proposing an architecture that dispenses with recurrence entirely. The paper was framed around machine translation, but the architecture proved general enough to reshape the entire field.

The core mechanism is **self-attention**, and the single most useful way to understand it is as a parallel hash table lookup.

For an input sequence of token embeddings, self-attention computes three matrices via learned linear projections:

- **Q** (queries): what each token is looking for
- **K** (keys): what each token advertises about itself
- **V** (values): the actual content each token provides

The attention computation is:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

Every token's query is compared against every other token's key via dot product. The softmax normalizes these scores into a probability distribution. The output for each token is a weighted sum of all value vectors, where the weights reflect how relevant each other token is.

The `sqrt(d_k)` scaling factor prevents dot products from growing too large in high dimensions, which would push the softmax into saturation (effectively making it a hard argmax rather than a soft weighting).

Think of it as: every element in your array queries every other element simultaneously, receives back a relevance-weighted blend of their values, and all of this happens in a single matrix multiplication -- fully parallelizable across the entire sequence. Where an RNN processes tokens one by one in $O(n)$ sequential steps, self-attention lets every token see every other token in $O(1)$ depth.

The cost is $O(n^2)$ in computation and memory, because every token attends to every other token. This tradeoff -- constant depth for quadratic cost -- is the central tension that drives most of the architectural innovation covered in this document.

Multi-Head Attention: Multiple Concurrent Queries

Rather than running a single attention function, the transformer runs multiple attention heads in parallel -- typically 8 to 96, depending on model size. Each head has its own learned Q, K, V projection matrices. The results are concatenated and linearly projected.

Think of it as running multiple concurrent queries against the same database, each looking for different types of relationships. One head might attend to syntactic dependencies (subject-verb agreement). Another might attend to semantic relationships (coreference). Another might track positional patterns (nearby tokens). The multi-head mechanism lets the model capture multiple relationship types simultaneously without forcing them into a single attention pattern.

Positional Encoding: Array Indexing for an Orderless Operation

Self-attention is *permutation-equivariant*: if you shuffle the input tokens, the output shuffles identically. The mechanism has no inherent notion of token order. This is a problem -- "the dog bit the man" and "the man bit the dog" would produce identical representations without position information.

Positional encoding solves this by attaching position metadata to each token, analogous to adding array indices to elements of an unordered set.

The original transformer used **sinusoidal encoding** -- sin/cos waves of varying frequencies added directly to token embeddings. This is parameter-free and theoretically extrapolates to unseen lengths. GPT-2 and BERT used **learned positional embeddings** -- a lookup table of position vectors trained alongside the model. These work well but cannot extrapolate beyond the training length.

The modern standard is **Rotary Position Embedding (RoPE)** (Su et al., 2021). RoPE rotates query and key vectors by an angle proportional to their position, so that the dot product between them naturally encodes *relative* distance. It is parameter-free, naturally relative rather than absolute, and critically -- as we will see later -- can be extended post-training to

support longer contexts than the model was originally trained on. RoPE is used by LLaMA, Qwen, Mistral, and most current models.

The Transformer Block

A complete transformer block consists of:

- 1. Multi-head self-attention
- 2. Add & normalize (residual connection + layer normalization)
- 3. Position-wise feed-forward network (two linear layers with a nonlinear activation)
- 4. Add & normalize

These blocks are stacked -- 12 for small models, 96+ for large ones. The residual connections are architecturally critical: the transformer's **residual stream** functions as a shared communication bus. Each attention head and feed-forward layer reads from and writes to this stream additively. This framing, developed by Elhage et al. (2021), enables compositional analysis: individual components can be studied in terms of what they read from and write to the stream, rather than as opaque matrix multiplications. Think of it as shared mutable state on a bus, where each layer reads the current state and writes its contribution back additively.

Three Architectures, One Winner

The original transformer used both an encoder (bidirectional self-attention over input) and a decoder (causal self-attention over output, plus cross-attention to the encoder). Three variants emerged:

Architecture	Attention Pattern	Pre-training Objective	Examples
-----	-----	-----	-----
Encoder-only	Bidirectional	Masked language modeling (MLM)	BERT, RoBERTa
Decoder-only	Causal (left-to-right)	Next-token prediction	GPT, LLaMA
Encoder-decoder	Bidirectional + causal	Span corruption / denoising	T5, BART

Decoder-only won for generation, and the reasons are instructive:

1. **Simplicity.** One architecture, one training objective (predict the next token), one inference procedure.
2. **Scaling efficiency.** All parameters serve a single computation path. Encoder-decoder splits capacity across two separate stacks.
3. **In-context learning.** The autoregressive framing naturally supports few-shot prompting -- the "encoder" is simply the prefix of the generation.
4. **Hardware alignment.** Causal attention's triangular mask maps efficiently to modern GPU kernels.

Every frontier LLM today -- GPT-4, Claude, Gemini, LLaMA, DeepSeek -- uses a decoder-only architecture. The encoder-only lineage (BERT, RoBERTa) remains important for embedding and classification tasks, but the generative frontier is exclusively decoder-only.

Tokenization: Variable-Length Encoding for Language

LLMs do not operate on characters or words but on **subword tokens**. This is variable-length encoding optimized for frequency, analogous to how UTF-8 uses fewer bytes for common characters and more for rare ones.

Byte Pair Encoding (BPE) iteratively merges the most frequent adjacent byte/character pairs. The string "lower" might be tokenized as ["low", "er"] if that merge was learned during tokenizer training. Vocabulary sizes range from ~32K (LLaMA) to ~200K (GPT-4o). Larger vocabularies reduce sequence length (fewer tokens per text) but increase the embedding matrix size.

Why tokenization matters for model behavior: a model's effective context length depends on tokens, not characters. A tokenizer trained primarily on English will fragment other languages into many small tokens, effectively shrinking context and degrading performance. Tokenizer artifacts also explain some puzzling model failures -- inconsistent splitting of numbers, inability to count characters, or struggling with words that happen to fall on unusual token boundaries.

Part 3: The Scaling Discovery

Scaling Laws: A Moore's Law Moment

In 2020, Kaplan et al. at OpenAI established that language model loss follows power-law relationships with three variables:

- $L(N) \sim N^{-0.076}$ (parameters)
- $L(D) \sim D^{-0.095}$ (dataset size in tokens)
- $L(C) \sim C^{-0.050}$ (compute in FLOP)

These trends held over seven orders of magnitude. The implication was profound: performance was predictable from resources. Double your compute, and you could predict how much the loss would drop. This was the closest thing machine learning had ever had to Moore's Law -- a reliable, quantitative relationship between investment and return.

Kaplan's key recommendation was that **model size matters more than data size** for a fixed compute budget. Train very large models on relatively modest data and stop before convergence.

Chinchilla: The Budget Allocation Correction

In 2022, Hoffmann et al. at DeepMind overturned Kaplan's allocation recommendation. After training 400+ models from 70M to 16B parameters on 5B to 400B+ tokens, they found that **model size and training tokens should scale equally**: for every doubling of parameters, double the training data.

The optimal ratio is approximately **20 tokens per parameter**. By this measure, GPT-3 (175B parameters trained on 300B tokens) was significantly undertrained -- it should have seen ~3.5 trillion tokens. Chinchilla (70B parameters, 1.4T tokens) outperformed GPT-3 (175B), Gopher (280B), and Megatron-Turing NLG (530B) despite being smaller, because its compute budget was allocated more efficiently.

Beyond Chinchilla: When the Objective Function Changes

The Chinchilla ratio (20:1 tokens-to-parameters) was optimal for a specific cost function: minimize loss for a fixed compute budget. But practitioners now optimize for a different objective: **minimize inference cost** for a target capability level. A small model that has been trained far beyond the Chinchilla optimum is cheaper to serve than a large model trained to the Chinchilla optimum, even though the total training compute is higher.

Practice has moved dramatically past Chinchilla:

- LLaMA 3 (8B parameters) trains on 15T+ tokens -- a ratio of **1,875:1** versus Chinchilla's 20:1.
- Qwen3-0.6B reaches **60,000:1**.
- Loss keeps decreasing. The models keep getting better.

The scaling law is correct -- loss does decrease predictably with compute -- but the objective function has changed from "minimize training cost" to "minimize serving cost," and this changes the optimal allocation completely.

What Scaling Laws Cannot Tell You

Scaling laws predict loss on the pre-training objective. They do **not** predict:

- **Emergent capabilities.** When will a model suddenly be able to do multi-step math? Scaling laws offer no answer.
- **Downstream task performance.** Loss is a necessary but insufficient metric for the capabilities that matter in practice.
- **Alignment properties.** Nothing about scaling laws predicts whether a model will be helpful, honest, or safe.
- **Qualitative capability transitions.** Models acquire capabilities suddenly during training -- specific abilities appear at particular training steps with sharp transitions, not gradual improvement. We cannot predict in advance when a capability will crystallize.

Part 4: The Hardware Reality

The Memory Wall

The single most important fact for understanding modern LLM architecture is this: **transformer workloads are memory-bandwidth-bound, not compute-bound.**

A modern GPU might have 2,000 TFLOPS of compute but can only feed data to those cores at 8 TB/s. At FP16 (2 bytes per element), that is 4 trillion elements per second. Many operations saturate the memory bus long before the compute units are fully utilized. The architecture of LLMs is, in large part, an exercise in constrained optimization against this hardware reality.

This is not an abstract concern. It determines which architectural choices succeed and which fail. GQA exists because KV-cache is the inference bottleneck. MLA exists because DeepSeek needed to serve a 671B parameter model affordably. FlashAttention is not an algorithmic improvement but an IO scheduling optimization. SwiGLU won as the default activation despite costing 50% more parameters per FFN layer because the compute-per-quality tradeoff favors it when bandwidth, not FLOPS, is the constraint. The entire efficient attention research program is driven by a hardware limitation, not by mathematical insight.

GPU Evolution

NVIDIA dominates LLM training hardware. The progression tells the story of the memory wall:

GPU	Year	HBM	Capacity	Bandwidth	FP16 TFLOPS	Key Innovation
----	----	----	-----	-----	-----	----- ----- -----

A100	2020	HBM2e	80 GB	2.0 TB/s	312	3rd-gen Tensor Cores
H100	2022	HBM3	80 GB	3.35 TB/s	990	Transformer Engine, FP8
H200	2024	HBM3e	141 GB	4.8 TB/s	990	Memory upgrade over H100
B200	2025	HBM3e	192 GB	8.0 TB/s	2250	Chiplet design, FP4/FP6, 5th-gen NVLink

Notice: from A100 to B200, compute grew ~7x (312 to 2250 TFLOPS), but bandwidth grew only ~4x (2.0 to 8.0 TB/s). The gap is widening. Each generation of GPU has more compute capacity it cannot feed.

Google's TPUs take a different approach -- systolic arrays purpose-built for tensor operations, tightly integrated with JAX/XLA and custom interconnect (ICI) that scales to thousands of chips. The latest Ironwood (v7) TPU delivers 4,614 TFLOPS with 192 GB HBM at 7.2 TB/s, in pods of 9,216 chips reaching 42.5 exaflops. Google trains its own models (Gemini family) exclusively on TPUs.

Why $O(n^2)$ Matters in Bytes, Not FLOPS

Standard self-attention computes an $N \times N$ attention matrix, where N is the sequence length. For $N = 128K$ tokens, that matrix has 16 billion entries. At FP16, that is 32 GB -- just for the attention scores of a single head in a single layer. With 64 heads and 80 layers, the total intermediate memory for attention alone is enormous.

But the real bottleneck is not storing this matrix -- it is the memory bandwidth required to read and write it. The attention computation itself is embarrassingly parallel and fits well on GPU compute cores. The problem is shuttling the data between HBM (high-bandwidth memory, the GPU's main memory) and SRAM (the on-chip cache, which is fast but tiny -- typically 20 MB per streaming multiprocessor on an H100).

This is exactly the same class of problem you encounter when optimizing matrix operations for CPU cache hierarchy: the algorithm is fine, but the memory access pattern is killing performance.

FlashAttention: Cache-Line Optimization for Attention

FlashAttention (Dao et al., 2022) changed **no math**. It computes exact standard attention. The innovation is purely about memory access patterns -- it is a cache-tiling optimization, the same technique that has been used in high-performance computing for decades.

The key insight: instead of materializing the full $N \times N$ attention matrix in HBM, tile the computation so that each tile fits in GPU SRAM. Compute attention for one tile, write the result, move to the next. By restructuring the memory access pattern, FlashAttention:

- Reduces memory usage from $O(N^2)$ to $O(N)$
- Achieves 2-4x wall-clock speedup
- Makes previously memory-prohibitive sequence lengths tractable

FlashAttention-2 (2023) improved parallelism, reaching 50-73% of theoretical max FLOPS on A100. FlashAttention-3 (2024) targeted the H100's asynchronous execution model, reaching 740 TFLOPS (~75% H100 utilization) and added FP8 paths.

FlashAttention is now the standard attention implementation in virtually all serious LLM training and inference stacks. Every programmer has optimized an algorithm by restructuring memory access patterns -- FlashAttention is that insight applied to the most expensive operation in deep learning.

GQA and MLA: Reducing the Working Set

The **KV-cache** is the memoization table for autoregressive generation. During generation, each new token attends to all previous tokens. Rather than recomputing all key/value projections from scratch, they are cached. For a 70B model with 128K context at FP16, the KV-cache alone can consume ~40 GB per sequence -- larger than the model weights themselves for long contexts.

This is why context length is an infrastructure problem, not just an algorithm problem. The cache grows linearly with sequence length, and it must be read for every token generated. The KV-cache, not the model weights, is the dominant memory cost for inference.

Grouped Query Attention (GQA) reduces the KV-cache by sharing key/value projections across groups of query heads. Standard multi-head attention gives each head its own K and V projections -- with 64 heads, that means 64 sets of keys and values to cache. GQA groups heads: LLaMA 2 70B uses 8 KV heads for 64 query heads (8:1 ratio), reducing the cache by 8x while preserving nearly all of MHA's quality. GQA is now the universal default for open models.

Multi-head Latent Attention (MLA), introduced by DeepSeek-V2, goes further. It compresses the full KV representation into a low-rank latent vector -- 16K-dimensional KV compressed to 512 dimensions per token. The KV-cache drops from ~214 GB to ~7.6 GB, a 28x reduction. The latent is projected back to full K/V at attention time. RoPE is applied to a separate decoupled component to preserve positional information through the compression. MLA is more aggressive than GQA but has so far been DeepSeek-specific; Kimi K2's adoption suggests broader uptake is possible.

Variant	Mechanism	KV Cache Cost	Quality vs MHA	Who Uses It
-----	----- -----	-----	-----	-----
MHA	Independent Q, K, V per head	$O(n_heads)$	Baseline	OLMo 2, GPT-3
MQA	Shared single K,V across all heads	$O(1)$	Slight loss	PaLM, Falcon
GQA	K,V shared across groups of heads	$O(n_groups)$	~MHA	Llama 3, Qwen, Mixtral
MLA	Low-rank latent compress K,V to d_c dim	$O(d_c)$	~MHA	DeepSeek V2/V3/R1

The KV-Cache as Virtual Memory: PagedAttention

The KV-cache for a serving system must manage memory for hundreds of concurrent requests, each with different sequence lengths that grow over time. Traditional systems pre-allocate contiguous memory blocks for each request, wasting 60-80% of KV-cache memory on fragmentation (allocated but unused space for sequences that might grow).

PagedAttention (Kwon et al., 2023) solves this with the same insight as OS virtual memory: store the KV-cache in non-contiguous fixed-size blocks mapped via a block table. This is literally page tables for attention state. PagedAttention reduces memory waste to under 4% and is the key insight that made vLLM the dominant open-source serving engine.

Training Parallelism

No single GPU can hold a frontier model. A 70B-parameter model at FP16 requires ~140 GB just for weights, plus optimizer states (2-3x that with Adam), gradients, and activations. Training distributes work across hundreds or thousands of GPUs using complementary parallelism strategies:

- **Data Parallelism (DP):** Each GPU holds a full model copy and processes different data batches. Gradients are averaged via AllReduce.
- **Tensor Parallelism (TP):** Splits individual layer weight matrices across GPUs. Requires extremely fast interconnect (NVLink) because communication happens within every forward/backward pass. Used within a single node.
- **Pipeline Parallelism (PP):** Splits the model's layers sequentially across GPU groups. GPU 0 runs layers 0-15, GPU 1 runs layers 16-31. Creates bubbles (idle time waiting for the previous stage) mitigated by micro-batching.
- **Expert Parallelism (EP):** For MoE models -- each expert resides on a different GPU. Requires All-to-All communication.

Real frontier training combines all three in **3D parallelism**: TP within a node (8 GPUs connected by NVLink at 900 GB/s to 1.8 TB/s), PP across nodes within a rack, DP across racks (connected by InfiniBand at 400-800 Gb/s per port). The Megatron-LM framework is the reference implementation.

ZeRO/FSDP eliminates the redundancy of data parallelism by partitioning optimizer states, gradients, and parameters across GPUs. Each GPU holds only 1/N of everything, gathering parameters on-demand via AllGather before each layer's computation. This is the default for PyTorch-based training.

Part 5: The Architecture Zoo

Modern Architectural Convergence

Despite the variety of model families, the architectural choices of frontier LLMs have converged remarkably. The standard recipe for a 2024-2025 model is:

- Decoder-only transformer
- RoPE positional encoding
- Pre-norm with RMSNorm
- SwiGLU activation in the feed-forward layers
- GQA (or MLA for DeepSeek)
- BPE tokenizer

The differences that matter are not in these choices but in training data, post-training recipes, and the decision between dense and Mixture-of-Experts architectures.

Mixture of Experts: Microservices for Neural Networks

In a standard (dense) transformer, every token passes through every parameter. In an MoE model, the feed-forward network in each transformer block is replaced by N expert FFNs plus a **router** -- a small learned network that dispatches each token to the top-k experts (typically k=1 or k=2). Only those experts' parameters are activated. This decouples total parameter count from per-token compute.

The analogy is precise: MoE is a load-balanced microservices architecture. A router dispatches each request (token) to 2 or 8 (or 16, or 256) specialist workers. The total workforce is large; the per-request cost is small. And the load-balancing problem is literally the same distributed systems problem -- without balancing, routers collapse to using a few experts, wasting the rest.

The Switch Transformer (Fedus et al., 2021) introduced an auxiliary loss penalizing uneven expert utilization. DeepSeek-V3 pioneered an auxiliary-loss-free balancing strategy, avoiding the quality degradation that auxiliary losses can cause.

Key MoE models:

Model	Total Params	Active Params	Experts	Routing	Notable Innovation
-----	-----	-----	-----	-----	----- ----- -
Mixtral 8x7B	46.7B	12.9B	8	Top-2	Proved MoE viability in open-source
DeepSeek-V3	671B	37B	256	Top-8	Shared experts, aux-loss-free balance
Llama 4	400B+	17B+	-	-	Meta's first MoE Llama
GPT-4 (?)	~1.8T (?)	~280B (?)	~16 (?)	-	Widely reported, never confirmed

The "how many parameters?" question is now ambiguous. A 671B MoE model using 37B active params per token has very different inference characteristics from a 671B dense model. MoE allows scaling knowledge capacity (total params) while controlling inference cost (active params). The trend in 2025 is clear: MoE is dominant for frontier models; dense persists for smaller models and where serving simplicity matters.

Dense vs. MoE Decision

Factor	Dense	MoE
-----	-----	----- ----
When to choose	Simpler training, lower infra needs	Need large capacity at lower inference cost

Compute per token	All params active	Fraction active (typ. 10-20% of total)
Memory at inference	Proportional to param count	Full model must be in memory despite sparsity
Training complexity	Standard	Load balancing, expert routing, auxiliary loss
Key tradeoff	Simple but expensive at scale	Cheap inference but hard to train and serve

State Space Models: Stream Processing for Sequences

State Space Models (SSMs) revisit the sequential processing that transformers abandoned -- but with crucial improvements that make them practical.

The core idea: model a sequence as a linear dynamical system, then either convolve (parallel, for training) or recur (constant memory, for inference). **Mamba** (Gu & Dao, 2023) made SSM parameters *input-dependent* ("selective" state spaces), allowing the model to decide per-token what to remember or forget.

Think of SSMs as stream processing: they maintain a fixed-size state that is updated as each token arrives, like a sliding window computation with learned update rules. Transformers, by contrast, are random-access -- they can attend to any position at any time, but the cost scales with sequence length. SSMs provide $O(1)$ memory per inference step by eliminating the KV-cache entirely.

Mamba-3B matched transformer performance at 2x its size on language modeling. Inference throughput is 5x higher than comparably-sized transformers. But no pure SSM model has matched frontier transformers at 70B+. The fundamental limitation: SSMs struggle with tasks requiring precise long-range retrieval from context, because their fixed-size state cannot perfectly preserve arbitrary positional information the way attention's full KV-cache can.

The practical outcome is **hybrid architectures** that interleave attention layers (for retrieval-critical positions) with SSM layers (for the bulk of computation). Jamba (AI21) and MoE-Mamba demonstrated this approach. The hypothesis: if most of what transformers compute in middle layers does not require full attention (plausible, given that attention patterns are often sparse), then SSM layers can handle it while attention layers handle retrieval. This is supported by infrastructure economics -- SSMs eliminate KV-cache, which is the dominant inference memory cost.

RWKV takes a different path: formulated so it can be trained like a transformer but run like an RNN. Models up to 14B parameters have been trained. Performance is competitive with transformers on standard benchmarks but trails on retrieval-heavy tasks -- the fundamental limitation of fixed-size recurrent state.

xLSTM (Beck et al., 2024) modernizes the LSTM with matrix-valued memory and exponential gating. At 350M parameters, xLSTM achieves competitive perplexity against Llama and Mamba.

Overall: none of these alternatives have replaced the transformer at frontier scale. The dominant trend is hybridization -- mixing attention with subquadratic methods to capture efficiency benefits while retaining attention's retrieval precision where it matters.

Context Extension

The original transformer used 512 tokens. GPT-3 used 2,048. Current models range from 8K to 10M+ tokens. Extending context requires:

- 1. Positional encoding that generalizes.** RoPE can be extended post-training by modifying its frequency spectrum. YaRN (Peng et al., 2023) combines NTK-by-parts scaling with an attention temperature factor, extending context with 10x fewer tokens and 2.5x fewer training steps than earlier methods.
- 2. Efficient attention.** FlashAttention makes long contexts tractable. Sliding window attention (used by Mistral 7B) bounds per-layer cost at $O(W)$ rather than $O(N)$. Ring Attention distributes the sequence across devices in a ring topology, overlapping communication with computation for near-linear scaling.
- 3. Sufficient training data at long contexts.** A model trained on short documents will not effectively use a long context window regardless of architectural support.

The critical distinction: **"supports" vs. "uses."** Many models advertise 1M+ context windows. Gemini 1.5 Pro reports 99.7% recall on needle-in-a-haystack at 1M tokens. Most open models show significant degradation on retrieval tasks beyond 32-64K tokens even when they technically accept longer inputs. The "lost in the middle" effect -- where models poorly attend to information in the middle of long contexts -- remains a practical limitation. Advertised context length and effective context length are different things.

Model Family Comparison

The following table captures the architectural state of the art as of March 2026:

Model Family	Type	Total Params	Active Params	Context	Attention	Pos Encoding
-----	----	-----	-----	-----	-----	-----
						-
GPT-4 / 4o	MoE	~1.8T (?)	~280B (?)	128K	MHA (?)	Unknown
Claude 3.5 / 4	? (?)	Undisclosed	Undisclosed	200K-1M	Unknown	Unknown
Gemini 1.5 / 2	MoE	Undisclosed	Undisclosed	1M-10M	Unknown	Unknown
Llama 3 / 3.1	Dense	8B/70B/405B	Same	128K	GQA	RoPE (500K base)
Llama 4	MoE	400B+	17B+	10M	GQA	RoPE
DeepSeek V3	MoE	671B	37B	128K	MLA	RoPE (decoupled)
Qwen 2.5	Dense	0.5B-72B	Same	128K	GQA	RoPE
Mixtral 8x7B	MoE	45B	~14B	32K	GQA	RoPE
Phi-4	Dense	14B	Same	16K	GQA	RoPE

Note the epistemic markers: (?) indicates unconfirmed leaks or external estimates, not official disclosure. Two of the most capable model families (GPT-4 and Claude) have essentially no public architectural disclosure.

What Actually Drives Quality

The evidence from years of model development points to a clear hierarchy:

Decision	Impact on Output Quality
-----	-----

Training data quality & scale	Dominant factor. Architecture is secondary to data.
RLHF / post-training recipe	Massive. The gap between base and aligned is enormous.
MoE vs Dense (at same FLOP)	Real. MoE wins on capacity per FLOP.
MLA vs GQA	Primarily efficiency. Quality is comparable.
SwiGLU vs GELU	~1-2% perplexity. Matters at scale, invisible to users.
RMSNorm vs LayerNorm	Training speed win. No measurable quality difference.

And the honest "we don't know why" list:

- **Why does more data keep helping?** Scaling laws predict it, but we lack a mechanistic explanation for why 15T tokens produces better results than 5T beyond "more patterns."
- **Why does SwiGLU help?** The gating mechanism helps empirically. The theoretical justification is post-hoc.
- **Why does RLHF work so well?** The theory of why preference optimization produces such dramatic improvement is incomplete.
- **Why do reasoning chains help?** Chain-of-thought produces massive gains. Whether it serves as a scratchpad for intermediate computation or merely shapes outputs toward correct answers is debated.

Inference Optimization

Two distinct phases with radically different compute profiles:

- **Prefill:** Process the full prompt in parallel. Compute-bound. High GPU utilization.
- **Decode:** Generate one token at a time. Memory-bandwidth-bound. Low arithmetic intensity.

This asymmetry drives serving optimizations:

Continuous batching (Orca, 2022) dynamically inserts new requests as old ones complete, keeping the GPU saturated. This alone delivers 3-10x throughput improvement over static batching.

Speculative decoding uses optimistic concurrency: a small draft model generates k candidate tokens cheaply, the large target model verifies all k in a single forward pass (parallel verification costs the same as generating one token). Accepted tokens are free throughput. For code -- which has predictable patterns like closing braces and boilerplate -- acceptance rates reach 75-85%, yielding 2-3x speedups.

Quantization compresses weights from FP16 (2 bytes) to INT8 (1 byte) or INT4 (0.5 bytes). Think of it as lossy compression for weights -- JPEG for neural networks. You choose your quality/size tradeoff:

Method	Bits	Best For
-----	----	-----
INT8	8	Production with <1% quality loss
GPTQ	4	GPU inference, well-studied
AWQ	4	GPU inference, faster quantization
GGUF	2-8	CPU/Apple Silicon inference

A critical practical finding: kernel implementation matters more than the quantization algorithm. Marlin-AWQ achieves 741 tok/s versus baseline FP16, because the kernel is optimized for memory access patterns, not because AWQ is theoretically superior.

Part 6: From Base Model to Useful Model

The Post-Training Gap

A base model trained on next-token prediction is, by default, an autocomplete engine. Given "The capital of France is", it will likely produce "Paris" -- but it will also happily continue with racist jokes, fabricated citations, or incoherent rambling, because all of these appear in its training data. The gap between a base model and a useful assistant is entirely a post-training phenomenon.

Post-training transforms autocomplete into an assistant through three major methods: RLHF (with PPO), DPO, and GRPO. Understanding what these do -- and what we do not understand about how they work -- is essential for anyone using LLMs in production.

RLHF: The Original Pipeline

Reinforcement Learning from Human Feedback proceeds in three stages:

Stage 1: Supervised Fine-Tuning (SFT). The base model is fine-tuned on curated (prompt, response) pairs. This teaches it the format of instruction-following: how to respond to questions, follow instructions, and maintain conversational structure.

Stage 2: Reward Model Training. A reward model learns to predict human preferences. Architecturally, it is the same pretrained LLM with the output layer replaced by a scalar head. Human annotators compare pairs of model responses and rank them. The reward model is trained via the Bradley-Terry pairwise comparison model:

$$L(\theta) = -E[\log(\text{sigmoid}(r_{\theta}(x, y_w) - r_{\theta}(x, y_l)))]$$

This maximizes the probability that the reward gap between chosen and rejected responses matches the human label.

Stage 3: PPO Optimization. This is the RL step. PPO fine-tunes the SFT model to maximize reward while staying close to the reference policy. It requires **four models loaded**

simultaneously:

Model	Role	Trainable
-----	-----	-----
Actor (Policy)	Generates responses; the model being optimized	Yes
Critic (Value)	Estimates expected future reward per token	Yes
Reward Model	Scores complete responses	No
Reference Model	Frozen SFT model; provides baseline log-probs	No

The KL divergence penalty is the critical stabilizer: $R_{\text{modified}} = R_{\text{reward}} - \beta * KL(\pi_{\text{actor}} || \pi_{\text{ref}})$. Without it, the policy rapidly drifts into regions that exploit reward model weaknesses -- generating responses that score high but are nonsensical or degenerate. This failure mode, **reward hacking**, is a constant concern. The KL penalty keeps the policy's output distribution close to the SFT model's.

Computational cost: RLHF training spends ~80% of compute on sample generation. The four-model requirement roughly doubles memory versus SFT alone. A single RLHF iteration over a 70B model takes ~330 seconds.

DPO: The Closed-Form Shortcut

Direct Preference Optimization (Rafailov et al., 2023) discovered that the constrained optimization problem RLHF solves with PPO has a **closed-form analytical solution**. By rearranging the math, you can derive the optimal policy directly from preference data without ever training a reward model or running RL.

The key mathematical insight: the optimal policy for the KL-constrained reward maximization problem can be expressed as:

$$\pi^*(y|x) = (1/Z(x)) * \pi_{\text{ref}}(y|x) * \exp(r(x,y) / \beta)$$

Inverting this to express reward in terms of the policy, and substituting into the Bradley-Terry model, the partition function cancels, yielding a supervised learning loss:

$$L_{\text{DPO}} = -E[\log(\text{sigmoid}(\beta * (\log(\pi_{\theta}(y_w|x)/\pi_{\text{ref}}(y_w|x)) - \log(\pi_{\theta}(y_l|x)/\pi_{\text{ref}}(y_l|x))))]$$

No RL loop, no reward model, no critic, no sampling during training. One model plus a frozen reference, trained with a standard cross-entropy-style loss on preference data.

DPO matches or exceeds PPO on summarization, dialogue, and sentiment tasks. It is roughly 2-4x cheaper than PPO and far simpler to implement. Its publication in mid-2023 democratized alignment -- before DPO, only organizations with significant RL infrastructure could perform alignment training. DPO made it accessible to anyone who could run supervised fine-tuning.

The tradeoffs: DPO can overfit to training preferences, exhibits a bias toward verbosity (longer outputs score higher in preference data), and cannot discover new behaviors through exploration since it trains on a fixed dataset.

GRPO: Group Relative Policy Optimization

GRPO, introduced by DeepSeek, occupies the middle ground: it uses RL (unlike DPO) but eliminates the critic network (unlike PPO). For each prompt, GRPO generates a **group** of G completions (typically 16-64), scores each with a reward function, and normalizes advantages relative to the group:

$$A_i = (r_i - \text{mean}(\text{group_rewards})) / \text{std}(\text{group_rewards})$$

This halves the memory needed for gradient-tracked parameters compared to PPO (no critic model). The tradeoff: GRPO needs many samples per prompt for stable advantage estimates, but generation is parallelizable.

The breakthrough came from pairing GRPO with **verifiable rewards** rather than learned reward models:

- **Math:** Check if the final answer matches ground truth (binary reward).
- **Code:** Execute generated code against test cases (binary pass/fail).
- **Format:** Check if output follows required structure.

This sidesteps reward model failure modes entirely. You cannot hack a compiler or a mathematical equality check. DeepSeek-R1 used only accuracy and format rewards, with no neural reward model.

The result was remarkable: DeepSeek-R1-Zero, trained with GRPO from a base model with no SFT, spontaneously developed chain-of-thought reasoning, self-verification, and backtracking. No one instructed it to reason this way. Optimizing for answer correctness, via RL with verifiable rewards, incentivized structured reasoning as an emergent behavior.

Dimension	PPO	DPO	GRPO
-----	-----	-----	-----
Models during training	4 (actor, critic, reward, ref)	2 (policy, ref)	2-3 (policy, ref, optional reward)
Reward signal	Learned RM	Implicit	Learned RM or verifiable
Online generation	Yes (1/prompt)	No	Yes (G/prompt)
Memory cost	Highest	Lowest	Middle
Best for	General alignment	Preference matching	Reasoning with verifiable answers

Constitutional AI

Anthropic’s Constitutional AI addresses the question: how do you scale safety training without requiring humans to label every harmful interaction?

Phase 1 (Self-Critique): Generate responses (including to adversarial prompts), ask the model to critique its own response against a written constitution of natural-language principles, revise the response, then fine-tune on the revised outputs.

Phase 2 (RLAIF): Generate response pairs, use an AI model (not humans) to judge which better satisfies the constitutional principles, train a preference model from these AI-generated preferences, then run standard RL using this preference model as reward.

The constitution is legible natural language -- you can read exactly what principles the model was trained to follow. Anthropic has published Claude’s constitution. The key insight:

harmlessness can be handled by AI feedback (cheaper, more consistent), while helpfulness is handled by human feedback (where human judgment matters more). This separation allows independent optimization of each axis.

The Elicitation Hypothesis

The deepest puzzle in alignment: post-training modifies a tiny fraction of the model's learned representations, yet produces dramatic behavioral changes. Weight changes from RLHF/DPO are small relative to pretraining -- yet the model goes from producing raw completions to following instructions, refusing harmful requests, and exhibiting apparent reasoning.

The dominant explanation: **post-training does not add capabilities; it elicits capabilities already present in the base model.** The base model, trained on trillions of tokens including dialogue, code, and reasoning chains, already "knows" how to do these things. It just does not know when to deploy them. Post-training teaches the model which behaviors to surface by default.

Supporting evidence: base models can be prompted into instruction-following behavior with careful few-shot examples, suggesting the capability exists pre-alignment. RLHF operates at the response level yet changes token-level generation, consistent with redirecting existing circuits rather than building new ones.

Why this works *so well* with *so little* change remains genuinely mysterious. The weight space is billions-dimensional; why does a small perturbation reliably shift behavior in the desired direction rather than in random directions? The geometric structure of the loss landscape that makes this possible is not understood.

Part 7: Beyond Text

Multimodal Architecture: How Models See

The foundational insight of vision transformers: treat an image like a sentence. A 224x224 image is split into 16x16 patches, yielding 196 patches. Each patch is flattened and linearly projected into an embedding vector, producing a sequence of "visual tokens" that a standard transformer processes identically to text tokens.

CLIP (OpenAI) trains a vision encoder and text encoder jointly on 400M image-text pairs using contrastive learning, producing image representations already aligned with language. **SigLIP** replaced CLIP's softmax contrastive loss with a sigmoid loss (each pair becomes an independent binary classification), eliminating the need for large batch sizes. SigLIP is now the default vision encoder for open-source vision-language models.

Three fusion strategies connect vision to language:

Late Fusion (Adapter). The vision encoder and LLM are separate pre-trained models connected by a small projection layer. Visual features are projected into the LLM's embedding space and concatenated with text tokens. LLaVA demonstrated this is remarkably effective: a two-layer MLP connecting CLIP ViT to LLaMA, trainable in ~24 GPU-hours. The limitation: cross-modal interaction is shallow. The LLM never learned to "see" -- it learned to interpret projected features.

Cross-Attention Fusion. Dedicated cross-attention layers where visual features serve as keys/values and text features serve as queries. More expressive than adapters, does not explode input sequence length. Used by Flamingo and reportedly by Claude.

Early Fusion (Native Multimodal). All modalities tokenized into a shared vocabulary and processed through the same transformer from layer 1. Used by Gemini (trained multimodal from scratch), GPT-4o, and Llama 4. Deepest cross-modal interaction but requires massive multimodal pre-training -- cannot be retrofitted onto an existing text-only model cheaply.

The practical implications are direct:

- Vision encoder quality matters more than LLM size for visual tasks.
- Resolution determines the detail ceiling. Models at 224x224 physically cannot read small text.

- Adapter-based models fail on deep spatial reasoning ("what is to the left of the red object?"). Native multimodal models handle these better.
- Token budgets for images vary dramatically (256 to 2,000+ per image), affecting how much context window remains for text.

Audio

Audio enters LLMs through two paths: continuous features (Whisper encoder representations projected into the LLM's embedding space) or discrete audio tokens (EnCodec compresses audio into ~75 tokens/second using residual vector quantization). GPT-4o processes audio tokens natively with ~320ms latency, preserving tone and emotion that speech-to-text pipelines lose. If your application cares about *how* something is said rather than just *what*, the architectural difference between pipeline and native processing is decisive.

Video

Video is the hardest modality: 30fps at 224x224 with 16x16 patches produces 5,880 tokens per second. Even a 10-minute video can produce 50K+ visual tokens. Solutions include uniform frame sampling, adaptive frame selection (VideoBrain trains the model to decide when it needs more frames), and temporal SSM encoding (STORM places a Mamba layer between the vision encoder and LLM for linear-cost temporal integration). Long-context models (Gemini's 10M window) have a structural advantage for video.

Code Generation: Same Architecture, Different Training

Code LLMs are standard transformer decoders trained on code-heavy data with code-specific objectives. No major code model uses a fundamentally different architecture.

Fill-in-the-Middle (FIM) is the key training innovation. Standard autoregressive training only predicts left-to-right. FIM teaches the model to generate a middle span given prefix and suffix. The document is rearranged: `<PRE> prefix <SUF> suffix <MID> middle`. This is critical for IDE completion where the cursor sits in the middle of a file.

The "free lunch" finding (Bavarian et al., 2022): FIM can be added to autoregressive training at up to 50% rate with no cost to left-to-right performance. Recent advances include AST-Aware FIM (selecting structurally meaningful spans using the Abstract Syntax Tree, +5

point improvement) and Horizon-Length Prediction (predicting how many tokens remain in the middle span, up to +24% improvement).

Code assistants in practice are systems, not single models. The architecture involves context gathering (current file, recently opened files, imported modules), speculative decoding for latency (small draft model generates candidates, large model verifies), and multi-model routing (small code-specific model for autocomplete, frontier model for chat-based generation).

What makes code models better follows the general pattern: training data quality dominates, followed by data composition, then scale. Architecture is secondary. FIM and objective design have measurable impact but are training innovations, not architectural ones. At frontier scale, general-purpose models (Claude, GPT-4) achieve strong code performance without being "code models" -- broader knowledge compensates for less code-specific optimization.

RAG and Retrieval Integration

Every production LLM deployment uses retrieval. RAG (Retrieval-Augmented Generation) addresses the fundamental limitation that LLM knowledge is frozen at training time and cannot be updated without retraining.

Embedding models map text to fixed-dimensional vectors (384-3072 dimensions) such that semantically similar texts produce vectors with high cosine similarity. Most are encoder-only transformers (BERT-family) using bidirectional attention -- architecturally distinct from decoder-only LLMs. The training signal is contrastive learning: push matching pairs together, push non-matching pairs apart.

The canonical RAG pipeline:

1. **Index.** Chunk documents, embed each chunk, store vectors in a database (Pinecone, FAISS, pgvector).
2. **Retrieve.** Embed the user query, perform approximate nearest-neighbor search to find top-K relevant chunks.
3. **Generate.** Concatenate retrieved chunks into the LLM prompt. The LLM generates a grounded answer.

Advanced RAG adds pre-retrieval processing (query rewriting, decomposition, HyDE), hybrid retrieval (dense vectors + sparse BM25 keyword search, fused via reciprocal rank fusion), and post-retrieval refinement (cross-encoder reranking, contextual compression).

The architectural difference between bi-encoders and cross-encoders is instructive: bi-encoders encode query and document independently (fast, scalable, but query and document tokens never interact), while cross-encoders concatenate them and process jointly (100-1000x slower, but significantly more accurate because full attention operates across the pair). Two-stage retrieval -- bi-encoder recall, cross-encoder reranking -- is now standard practice.

Long context vs. RAG is a false dichotomy. Gemini's 10M token context window does not eliminate the need for RAG:

Scenario	Better Approach	Why
-----	-----	-----
Single long document, full comprehension	Long context	Summarization requires seeing everything
Large corpus, specific factual questions	RAG	Retrieve the needle; avoid paying for the haystack
Dynamic/updating knowledge base	RAG	Re-embed new documents; no need to re-prompt
Cost-sensitive, high-concurrency serving	RAG	Dramatically lower per-request compute

The practical trend is **RAG within long context**: retrieve the most relevant chunks, then place them in a long-context window for the model to reason over simultaneously.

Several research lines have attempted to build retrieval directly into the transformer architecture. RETRO (DeepMind) interleaves standard attention with chunked cross-attention over retrieved neighbors, matching GPT-3 on the Pile with 25x fewer parameters. kNN-LM interpolates the LM's next-token distribution with a kNN lookup over a datastore at inference time, with no architectural changes. Despite strong results, external RAG dominates production because it works with any LLM, supports index management (updates, filtering, permissions), and does not require architectural modification of API-served models.

Constrained Decoding and Tool Use

Function calling is fine-tuning, not architecture. Models that support tool use are fine-tuned to recognize when a tool should be invoked and emit structured JSON describing the call. The

tool definitions, prompts, and responses are all flattened into a single text stream -- there is no separate architectural pathway.

Constrained decoding guarantees valid structured output at the decoding level: at each generation step, a state machine (finite automaton for JSON, pushdown automaton for CFGs) masks tokens that would violate the schema. The remaining valid tokens are renormalized. This is architecturally significant: the model's generation is a collaboration between the neural network (producing token probabilities) and a symbolic constraint system (filtering invalid tokens). It is a practical instance of neuro-symbolic integration, and it can actually *improve* speed by reducing the sampling space.

Knowledge Distillation

A large "teacher" model's knowledge is transferred to a smaller "student" model by training the student on the teacher's output distribution (soft labels) rather than hard labels. The soft labels contain richer information about inter-class relationships. Methods include logit-based distillation (minimize KL-divergence to teacher outputs), feature-based distillation (match intermediate representations), and data-free distillation (teacher generates synthetic training data for the student).

Notable examples: DistilBERT (66M from 110M BERT, retaining 97% performance), LLaMA 3.2 1B/3B (distilled from 8B/70B). The relationship between model size and knowledge capacity is not linear -- a 7B model trained on 15T tokens can outperform a 70B model trained on 1.4T tokens. Distillation exploits this: the teacher provides a compressed curriculum that lets the student learn more efficiently per parameter than from raw data alone.

Part 8: What We Don't Know

The Central Mystery: Why Does Next-Token Prediction Produce Reasoning?

The training objective is purely statistical: predict the next token. Yet sufficiently large models exhibit multi-step reasoning, planning, and abstraction. No satisfying mechanistic answer exists. Several hypotheses circulate:

- **Implicit world models:** Transformers may reconstruct latent variables of the data generation process, building internal representations that function as world models.
- **Compression forces generalization:** The pressure to compress vast training data into fixed parameters may force the discovery of general algorithms.
- **Circuit sharing and pre-caching:** Features learned for prediction may be reused compositionally for higher-order tasks.

None of these are mechanistically verified at the level of "here is the circuit that implements reasoning step X." The gap between statistical prediction and apparent cognition is the central open question of the field. This is not a detail to be filled in later -- it is a fundamental gap in scientific understanding.

What Mechanistic Interpretability Has Found

Concrete progress exists, but it covers a tiny fraction of what models compute.

Sparse Autoencoders (SAEs) can decompose polysemantic neurons (neurons that respond to multiple unrelated concepts) into interpretable monosemantic features. Anthropic's work on Claude 3 Sonnet extracted tens of millions of features that are multilingual, multimodal, and abstract -- including safety-relevant features (deception, sycophancy, bias) usable for model steering.

Induction heads (Olsson et al., 2022) are a fully understood mechanism: given `[ ..., A, B, ..., A ]`, an induction head attends from the second `A` back to the first and predicts `B`. This is a two-head circuit (previous-token head + induction head) that accounts for a significant fraction of in-context learning in small models. It remains one of the cleanest

examples of a fully reverse-engineered transformer computation.

Circuit tracing (Anthropic, 2025) introduced cross-layer transcoders that produce "replacement models" where building blocks are sparse, interpretable features. Applied to Claude 3.5 Haiku, this revealed that for poem generation, the model engages in forward-and-backward planning: identifying rhyming target words before constructing lines leading to them.

These are real, verified findings. But they are the equivalent of having a partial disassembly of a few subroutines with no source code, no architecture document, and no spec for most of the system's behavior.

The Specific Unknowns

How does in-context learning actually work? Beyond induction heads (which handle verbatim pattern matching), the mechanism for general in-context learning in large models is poorly understood. Theoretical work shows linear transformers can implement gradient descent on in-context examples, but this does not explain how 100B+ parameter models do arbitrary task adaptation from a few examples.

What happens in the middle layers? Early layers extract low-level features. Late layers specialize toward output prediction. The middle layers -- where the majority of computation occurs -- remain largely opaque. These layers presumably perform the bulk of reasoning, composition, and abstraction, but we lack tools to characterize what they compute.

What does RLHF mechanistically do to weights? We know the algorithm. We do not know what it does to internal representations at the feature level. Does RLHF create new circuits? Suppress existing ones? Reroute information flow? We know the behavioral effect without understanding the internal mechanism.

Are emergent abilities real or measurement artifacts? Wei et al. (2022) claimed certain capabilities appear suddenly at specific model scales. Schaeffer et al. (2023) argued the apparent discontinuity is an artifact of nonlinear evaluation metrics. The debate is unresolved. If emergence is real, we cannot predict when dangerous capabilities will appear.

Is chain-of-thought thinking or performance? CoT improves accuracy on reasoning tasks. But evidence for unfaithfulness is substantial: models generate rationalizations for biased answers without mentioning the bias. Anthropic found Claude 3.7 Sonnet with thinking had only 0.04% unfaithfulness rate on tested scenarios, but the causal role of CoT -- whether tokens in the chain are read back and used, or whether the computation happens in the forward pass regardless -- is not settled.

The Interpretability-Architecture Disconnect

A significant meta-finding: **interpretability research and architecture design are almost completely disconnected**. Circuit tracing reveals that models plan backward for poetry and use induction heads for pattern matching. But none of this informed the design of GQA, MLA, MoE routing, or RoPE. Architecture decisions are empirical -- we don't know why SwiGLU helps. Interpretability is retrospective analysis of systems designed by trial and error.

The one exception: the residual stream view explains *why* pre-norm works better than post-norm -- it preserves the additive structure that enables compositional analysis. This is one of the rare cases where interpretability and architecture connect.

Safety Unknowns

Sleeper agents (Hubinger et al., 2024) demonstrated that models can be trained with backdoor behaviors that persist through standard safety training. Linear probes on residual stream activations can detect defection with >99% AUROC. What remains unknown: whether such deceptive behaviors could arise *naturally* from training.

Jailbreaks work reliably across models, suggesting that safety training creates a relatively thin behavioral layer over base capabilities. The fact that simple prompt manipulations can override extensive safety training is not well explained at the circuit level.

Representation engineering (Zou et al., 2023) demonstrates that directional vectors corresponding to concepts like honesty or harmfulness can be extracted from activations and used to steer model behavior. But what it means for a model to have a "honesty direction" in activation space -- the representational and causal status of these vectors -- is philosophically and empirically disputed.

Part 9: Practical Guide

Model Selection Realism

The architecture taxonomy is informative for understanding **cost and efficiency** tradeoffs -- MoE vs. dense, GQA vs. MLA, quantization levels. But it tells you almost nothing about **quality** for your specific use case. The reason: training data and RLHF recipe are the dominant quality factors, and both are proprietary for every frontier model. Model selection for production use cannot be done from first principles. You must benchmark on your specific task.

Cost Predictability vs. Quality Unpredictability

From architecture, you can predict:

- Inference cost (active parameters, KV-cache size, quantization level)
- Memory requirements (total parameters, context length, batch size)
- Latency bounds (model size, hardware, speculative decoding acceptance rate)

From architecture, you cannot predict:

- Whether the model will be good at your task
- Whether it will handle edge cases gracefully
- Whether its failure modes will be acceptable
- How it will compare to a competitor's model

The Open-vs-Closed Transparency Gap

Model Family	Weights	Training Data	Training Recipe	Paper	Reproducibl e?
-----	-----	-----	-----	-----	-----

GPT-4 / 4o	No	No	No	Minimal report	No
Claude 3.5 / 4	No	No	No	None	No
Gemini 1.5 / 2	No	No	No	Partial	No
Llama 3	Yes	No	Partial	Yes	Partial
DeepSeek V3	Yes	No	Detailed	Yes	Best-in-class
Qwen 2.5	Yes	No	Partial	Yes	Partial

No frontier model is fully reproducible. Training data is never released. DeepSeek V3 comes closest by publishing detailed training recipes, FP8 training methodology, and auxiliary loss formulations. True full reproduction (data + code + weights + training logs) exists only for non-frontier models like OLMo (AI2).

"Open weights" means you can run inference and fine-tune. It does not mean you can reproduce the training. "Open source" in the LLM context almost never means what it means in software engineering.

Training Costs

Model	Estimated Cost	Hardware
-----	-----	-----
GPT-3 (175B)	~\$4.6M	10K A100s
GPT-4	\$78-100M	Rumored 20-25K A100s
Gemini Ultra	~\$191M	TPU v4 pods
LLaMA 3 405B	~\$30M+	16K H100s
DeepSeek-V3	\$5.6M	2.79M H800 GPU-hours

DeepSeek-V3's cost is notable: it used H800s (export-restricted H100 variants with reduced interconnect), demonstrating that MoE architectures and engineering efficiency can dramatically reduce costs. The dominant cost structure is hardware amortization (40-50%) and personnel (20-30%); electricity is 2-6%.

Inference Economics

Inference cost scales with tokens generated, KV-cache memory per concurrent request, and GPU utilization. Continuous batching and quantization are the primary levers. A well-optimized serving stack can reduce per-token cost by 5-10x over naive deployment. OpenAI's 2024 inference spend reached \$2.3B -- 15x GPT-4's training cost -- suggesting that for frontier reasoning models, inference cost now dominates.

A Programmer's Reference Table

LLM Concept	Programming Analogy
-----	-----
Self-attention	Parallel hash table lookup where every element queries every other element
KV-cache	Memoization of previous computation, growing linearly with sequence length
MoE routing	Load balancer dispatching requests to specialized microservices
Residual stream	Shared mutable state on a bus; each layer reads and writes additively
FlashAttention	Cache-oblivious algorithm / tiling optimization for memory hierarchy
Positional encoding	Array index metadata attached to elements of an unordered set
Tokenization	Variable-length encoding (like UTF-8) optimized for frequency
Quantization	Lossy compression (JPEG for weights) -- choose your quality/size tradeoff
PagedAttention	Virtual memory with page tables for KV-cache allocation
Speculative decoding	Optimistic concurrency: draft, then verify in batch

References

Foundational Papers

- Vaswani et al. (2017) "Attention Is All You Need"
- Bahdanau, Cho & Bengio (2014) "Neural Machine Translation by Jointly Learning to Align and Translate"
- Hochreiter & Schmidhuber (1997) "Long Short-Term Memory"
- Cho et al. (2014) "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation"
- Mikolov et al. (2013) "Efficient Estimation of Word Representations in Vector Space" [Word2Vec]
- Pennington, Socher & Manning (2014) "GloVe: Global Vectors for Word Representation"

GPT and BERT Lineages

- Radford et al. (2018) "Improving Language Understanding by Generative Pre-Training" [GPT-1]
- Radford et al. (2019) "Language Models are Unsupervised Multitask Learners" [GPT-2]
- Brown et al. (2020) "Language Models are Few-Shot Learners" [GPT-3]
- Devlin et al. (2018) "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding"
- Liu et al. (2019) "RoBERTa: A Robustly Optimized BERT Pretraining Approach"
- He et al. (2020) "DeBERTa: Decoding-enhanced BERT with Disentangled Attention"
- Raffel et al. (2019) "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" [T5]
- Lewis et al. (2019) "BART: Denoising Sequence-to-Sequence Pre-training"

Scaling Laws

- Kaplan et al. (2020) "Scaling Laws for Neural Language Models"
- Hoffmann et al. (2022) "Training Compute-Optimal Large Language Models" [Chinchilla]

Training and Optimization

- Loshchilov & Hutter (2017) "Decoupled Weight Decay Regularization" [AdamW]
- Sennrich, Haddow & Birch (2016) "Neural Machine Translation of Rare Words with Subword Units" [BPE]
- Kudo & Richardson (2018) "SentencePiece"
- Micikevicius et al. (2018) "Mixed Precision Training"

Position Encoding

- Su et al. (2021) "RoFormer: Enhanced Transformer with Rotary Position Embedding" [RoPE]
- Press et al. (2022) "Train Short, Test Long: Attention with Linear Biases" [ALiBi]
- Peng et al. (2023) "YaRN: Efficient Context Window Extension of Large Language Models"

Efficient Attention

- Dao et al. (2022) "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness"
- Dao (2023) "FlashAttention-2"
- Dao et al. (2024) "FlashAttention-3"
- Ainslie et al. (2023) "GQA: Training Generalized Multi-Query Transformer Models"
- Shazeer (2019) "Fast Transformer Decoding: One Write-Head is All You Need" [MQA]
- Liu et al. (2023) "Ring Attention with Blockwise Transformers for Near-Infinite Context"

Alternative Architectures

- Gu & Dao (2023) "Mamba: Linear-Time Sequence Modeling with Selective State Spaces"

- Dao & Gu (2024) "Transformers are SSMs" [Mamba-2]
- Peng et al. (2023) "RWKV: Reinventing RNNs for the Transformer Era"
- Poli et al. (2023) "Hyena Hierarchy: Towards Larger Convolutional Language Models"
- Beck et al. (2024) "xLSTM: Extended Long Short-Term Memory"

Mixture of Experts

- Fedus et al. (2021) "Switch Transformers"
- Jiang et al. (2024) "Mixtral of Experts"
- DeepSeek-AI (2024) "DeepSeek-V3 Technical Report"

Model Families

- Touvron et al. (2023) "LLaMA: Open and Efficient Foundation Language Models"
- Touvron et al. (2023) "Llama 2: Open Foundation and Fine-Tuned Chat Models"
- Dubey et al. (2024) "The Llama 3 Herd of Models"
- Google (2024) "Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens"
- Abdin et al. (2024) "Phi-3 Technical Report"

Post-Training and Alignment

- Rafailov et al. (2023) "Direct Preference Optimization" [DPO]
- DeepSeek-AI (2025) "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL"
- Bai et al. (2022) "Constitutional AI: Harmlessness from AI Feedback"

Interpretability

- Elhage et al. (2021) "A Mathematical Framework for Transformer Circuits"
- Olsson et al. (2022) "In-context Learning and Induction Heads"
- Bricken et al. (2023) "Towards Monosemanticity"

- Templeton et al. (2024) "Scaling Monosemanticity"
- Anthropic (2025) "Circuit Tracing: Revealing Computational Graphs in Language Models"
- Schaeffer et al. (2023) "Are Emergent Abilities of Large Language Models a Mirage?"
- Wei et al. (2022) "Emergent Abilities of Large Language Models"
- Hubinger et al. (2024) "Sleeper Agents"
- Zou et al. (2023) "Representation Engineering"

Infrastructure

- Shoeybi et al. (2019) "Megatron-LM"
- Rajbhandari et al. (2020) "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models"
- Kwon et al. (2023) "Efficient Memory Management for Large Language Model Serving with PagedAttention"
- Yu et al. (2022) "Orca: A Distributed Serving System for Transformer-Based Generative Models"
- Leviathan et al. (2023) "Fast Inference from Transformers via Speculative Decoding"

Quantization and Compression

- Frantar et al. (2023) "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers"
- Lin et al. (2024) "AWQ: Activation-aware Weight Quantization"

Multimodal

- Dosovitskiy et al. (2020) "An Image is Worth 16x16 Words" [ViT]
- OpenAI CLIP (2021)
- Liu et al. (2023) "Visual Instruction Tuning" [LLaVA]

Code Generation

- Guo et al. (2024) "DeepSeek-Coder"
- Lozhkov et al. (2024) "StarCoder 2"
- Bavarian et al. (2022) "Efficient Training of Language Models to Fill in the Middle" [FIM]

Retrieval

- Lewis et al. (2020) "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"
- Borgeaud et al. (2022) "Improving Language Models by Retrieving from Trillions of Tokens" [RETRO]
- Khandelwal et al. (2020) "Generalization through Memorization: Nearest Neighbor Language Models" [kNN-LM]
- Liu et al. (2023) "Lost in the Middle"